

Heuristics for Planning — Q1 Notes

The key insight

In ordinary search, heuristics have to be supplied by a human who understands the domain. In planning the state and actions are described *declaratively* — each operator has explicit preconditions, add-list, delete-list — so heuristics can be **derived automatically** from the problem description. This is what makes planning heuristics distinctive.

A generic recipe:

Drop part of the problem to get an easier (“relaxed”) problem whose optimal cost is a lower bound on the original. Use that cost as h .

Provided the relaxation is genuinely easier (preferably polynomial), h is cheap and admissible.

1. Relaxation heuristics (the big ones)

Ignore preconditions

Drop all preconditions: every action is always applicable. The optimal relaxed plan length is the minimum number of actions whose add-lists cover the goal — a set-cover problem. Useful but loose.

Ignore delete lists (delete-relaxation)

Drop every action’s delete-list, so nothing ever becomes false. The relaxed problem is monotone and solvable in polynomial time. The relaxed plan length is h^+ .

Why it works: once a goal literal is achieved it stays achieved, so the planner never has to backtrack. **FastForward (FF)** uses exactly this as its heuristic — one of the most successful planning systems.

Subgoal independence

Plan for each conjunct of the goal in isolation. Then combine:

- $h_{\max}(s) = \max_{g \in \text{goal}} \text{cost}(g)$ — admissible, conservative.
- $h_{\text{add}}(s) = \sum_{g \in \text{goal}} \text{cost}(g)$ — not admissible (double-counts), but usually more informative.

Assumes subgoals don’t interact — a useful lie.

2. Use the structure of the goal

Landmarks. A landmark is a fact (or action) that must hold in *every* solution. Found by analysis of the goal and operators. They split planning into “before-landmark” and “after-landmark” phases and give a lower bound = number of landmarks not yet achieved.

Helpful actions. At each search node, only consider actions that appear in the relaxed plan from that node. Trades completeness for branching factor — usually a big win.

Means–ends analysis. Pick an unmet goal literal g ; pick an action whose add-list contains g ; recurse on its preconditions. Old idea (GPS), still informs how heuristics rank actions.

3. Shrink the search space

Repeated-state detection. Planning states are explicit sets of fluents — equality is cheap to test. A closed-list / transposition table avoids exploring the same state twice. Big win for breadth-first style planning.

Partial-order planning. Only commit to an ordering between actions when there’s a causal reason. Avoids exploring the $n!$ permutations of n independent actions.

State abstraction / pattern databases. Project the state onto a subset of fluents; pre-compute optimal cost in that abstract space; use as h . Admissible by construction.

4. Decompose the problem

Hierarchical / HTN planning. Plan with abstract operators first, then refine. Cuts the effective depth: planning a 1000-step low-level plan becomes planning a 10-step high-level plan, each step refined separately.

Goal serialisation. If two subgoals can be ordered such that solving the first never breaks the second (an “ordered” goal set), solve them one at a time. Finding such an ordering is itself a heuristic.

5. Reuse what you’ve already solved

Plan reuse / case-based planning. Store solved plans; on a new problem, retrieve a similar past plan and adapt. Costly to maintain but pays off in repeated domains.

Macro-operators. Compile frequently used action sequences into single operators (e.g. “open-and-walk-through-door”). Reduces tree depth at the cost of branching factor.

What to write in the exam answer

A strong answer mentions **at least one heuristic from each of three categories**:

1. A **relaxation** heuristic (ignore preconditions / ignore deletes / subgoal independence) — and note that the planner can derive these *automatically* from the operator descriptions; that’s the whole point.
2. A **structural** heuristic (landmarks, helpful actions, means–ends) — which exploits the goal’s syntactic structure.
3. A **search-space** or **decomposition** idea (partial-order planning, HTN decomposition, repeated-state detection) — which shrinks the tree rather than guiding it.

End by linking back to the question: planning’s edge over generic search is that operator descriptions expose enough problem structure for the planner to **compute its own heuristic**, rather than relying on a hand-written one.